



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



AWS Cloud Deployment and Infrastructure Automation

CSS7000 INTERNSHIP REPORT

Submitted by

UMESH - 20241CIT0176

Under the guidance of,

Dr.Ramesh Sengodan

**BACHELOR OF TECHNOLOGY
IN**

**COMPUTER SCIENCE AND ENGINEERING
(Internet Of Things)**



GAIN MORE KNOWLEDGE
REACH GREATER HEIGHTS

PRESIDENCY UNIVERSITY

BENGALURU

AUGUST 2026



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



PRESIDENCY SCHOOL OF ARTIFICIAL INTELLIGENCE & ADVANCED COMPUTING BONAFIDE CERTIFICATE

Certified that this report **AWS Cloud Deployment and Infrastructure Automation** is a bonafide work of **UMESH (20241CIT0176)**, who has successfully carried out the internship work and submitted the report for partial fulfilment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in **PRESIDENCY SCHOOL OF ARTIFICIAL INTELLIGENCE & ADVANCED COMPUTING, INTERNET OF THINGS** During 2026-2027.

Dr. Ramesh Sengodan

Professor

Internship Guide

PSAIAC

Presidency University

Mr. Francis Vijaykumar

Annareddy

Assistant Professor

Program Internship Coordinator

PSAIAC

Presidency University

Dr. Geetha Arjunan

Associate Professor

School Internship Coordinator

PSAIAC

Presidency University

Dr. Sampath AK

Professor & Head of the Department

PSAIAC

Presidency University

Dr. Shakkeera L

Dean

PSAIAC

Presidency University

Sl.No	Name of the Examiner	Signature
1	Mr. Tanveer Ahmed	



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



PRESIDENCY SCHOOL OF ARTIFICIAL INTELLIGENCE & ADVANCED COMPUTING DECLARATION

I am a student of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING (INTERNET OF THINGS), at Presidency University, Bengaluru, named **Umesh**, hereby declare that the internship work **AWS Cloud Deployment and Infrastructure Automation** has been independently carried out by me and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2026-2027. Further, the matter embodied in the internship has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

Umesh

USN: 20241CIT0176

Signature

PLACE: BENGALURU

DATE:

INTERNSHIP COMPLETION CERTIFICATE



ACKNOWLEDGEMENT

For completing this internship work, I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. I extend my gratitude to our beloved Chancellor, Pro-Vice Chancellor, and Registrar for their support and encouragement in completion of the internship.

I would like to sincerely thank my internal guide Dr. Ramesh Sengodan Professor, Presidency School Of Artificial Intelligence & Advanced Computing Presidency University, for her moral support, motivation, timely guidance and encouragement provided to me during the period of internship work.

I am also thankful to Dr. Sampath A.K., Professor, Head of the Department, Presidency School Of Artificial Intelligence & Advanced Computing, Presidency University, for his mentorship and encouragement.

I express my cordial thanks to **Dr. Shakkeera L**, Dean, presidency School Of Artificial Intelligence & Advanced Computing, Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my internship work.

We are grateful to **Dr. Geetha Arjunan**, Associate Professor, School Internship Coordinator, **Mr. Francis Vijay Kumar Annareddy**, Assistant Professor, Program Internship Coordinator, Presidency School Of Artificial Intelligence & Advanced Computing, Presidency University for facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

I am also grateful to Teaching and Non-Teaching staff of Presidency School Of Artificial Intelligence & Advanced Computing and also staff from other departments who have extended their valuable help and cooperation.

Umesh – 20241CIT0176

ABSTRACT

This project, titled **AWS Cloud Deployment and Infrastructure Automation**, presents the practical deployment and management of cloud-based infrastructure using Amazon Web Services (AWS). Cloud computing enables organizations to access computing resources such as storage, servers, networking, and application services without depending entirely on physical hardware. AWS provides a wide range of services that help developers and organizations build, deploy, and maintain applications in a cloud manner. The main objective of this project is to gain practical knowledge of AWS cloud services and understand the basic process of deploying applications in a cloud environment. The project consists of four important tasks. The first task involves deploying a static website on Amazon Simple Storage Service (Amazon S3). In this task, website files such as HTML, CSS, and JavaScript are uploaded to an S3 bucket, static website hosting is enabled, and the website is made accessible through a public URL.

The second task involves deploying a virtual machine using Amazon Elastic Compute Cloud (Amazon EC2) and installing a web server. An Ubuntu-based EC2 instance is launched and configured with suitable security rules. A web server such as Apache or Nginx is installed on the virtual machine, and a customized web page is created and accessed through the public IP address of the instance.

The third task focuses on creating a serverless function using AWS Lambda. The function is developed using Python or Node.js and is designed to return a JSON response. The function is tested using the AWS Management Console and connected to an API endpoint using Amazon API Gateway. This task demonstrates how applications can execute code without requiring the continuous management of servers.

The fourth task involves deploying a Docker container on a cloud virtual machine. Docker is installed on the EC2 instance, and a container is created using a suitable image such as Nginx or a Node.js application. The required port is exposed to the internet so that the application running

web link. The completion of these tasks provides practical experience in cloud storage, virtual machines, web server configuration, serverless computing, API development, containerization, networking, and security configuration. The project also demonstrates how different AWS services can work together to support modern application deployment. Overall, this project develops a strong foundation in cloud computing, DevOps practices, application hosting, and infrastructure automation. It also shows how cloud platforms can improve application availability, scalability, flexibility, and deployment.

TABLE OF CONTENTS

SL.NO	TITLE	PAGE.NO
	ACKNOWLEDGEMENT	v
	ABSTRACT	vi
	TABLE OF CONTENTS	viii
1	INTRODUCTION	9
2	OBJECTIVES	10
3	WORK PROGRESS	13
4	INTERNSHIP OUTCOME	35
5	RESULT	38

6	CONCLUSION	41
7	REFERENCES	42

INTRODUCTION

Cloud computing has become an important technology for developing, deploying, and managing modern applications. It provides on-demand access to computing resources such as storage, virtual machines, networking, databases, and serverless services through the internet. Amazon Web Services (AWS) is one of the most widely used cloud platforms because it offers flexible, scalable, secure, and cost-effective services for organizations and developers.

The project titled ‘AWS Cloud Deployment and Infrastructure Automation’ focuses on understanding and implementing essential AWS cloud services through practical deployment tasks. The project demonstrates how a static website can be hosted using Amazon Simple Storage Service (Amazon S3), how a virtual machine can be created and configured using Amazon Elastic Compute Cloud (Amazon EC2), how serverless computing can be implemented using AWS Lambda, and how a Docker container can be deployed on a cloud virtual machine.

The main purpose of this project is to gain practical knowledge of cloud infrastructure deployment and automation. Each task represents a common real-world cloud deployment process. The project also provides experience in configuring security rules, managing cloud resources, installing web servers, creating application endpoints, running containers, and testing deployed services through public URLs.

Through these activities, the project explains how AWS can reduce the need for physical infrastructure while improving application availability, scalability, and operational efficiency. It also introduces important concepts such as cloud storage, virtual machines, serverless architecture, containerization, networking, and access control. The successful completion of these tasks provides a strong foundation for further learning in cloud computing, DevOps, infrastructure automation, and application deployment.

OBJECTIVES

The main objective of the project titled 'AWS Cloud Deployment and Infrastructure Automation' is to understand the fundamental concepts of cloud computing and gain practical experience in deploying and managing applications through Amazon Web Services. Cloud computing has changed the way organizations use computing resources by providing flexible access to storage, servers, networking, and application services through the internet. Therefore, this project is designed to provide practical knowledge of how cloud resources can be created, configured, secured, and used for application deployment. It also helps in understanding how cloud platforms reduce the dependency on physical infrastructure and improve the availability, flexibility, and scalability of applications.

One of the important objectives of this project is to deploy a static website using Amazon Simple Storage Service, commonly known as Amazon S3. This objective includes creating an S3 bucket, configuring the required settings, uploading website files such as HTML, CSS, and JavaScript, and enabling static website hosting. The project also aims to make the website accessible through a public URL by applying the required permissions and bucket configuration. Through this activity, the working principles of cloud storage, object management, public access, and static website hosting can be understood in a practical manner.

Another objective is to create and configure a virtual machine using Amazon Elastic Compute Cloud, or Amazon EC2. This includes launching an Ubuntu-based virtual machine, selecting the required instance configuration, connecting to the instance, and configuring suitable security group rules. The project aims to provide practical experience in installing and managing a web server such as Apache or Nginx on the virtual machine. It also includes creating a customized web page and accessing that page through the public IP address of the EC2 instance. This task helps in understanding virtual machines, operating system configuration, firewall rules, ports, IP addresses, and server administration.

The project further aims to implement serverless computing by creating a function using AWS Lambda. The objective is to develop a simple function using Python or Node.js that returns a JSON response when it is executed. The function is tested through the AWS Management Console and can be connected to a public endpoint using Amazon API Gateway. This activity helps in understanding how code can be executed without continuously managing a physical or virtual server. It also introduces the concepts of event-driven execution, API communication, automatic scaling, and serverless application development.

The final objective is to understand containerization by deploying a Docker-based web application on a cloud virtual machine. This includes installing Docker, downloading or creating a suitable container image, running the container, mapping the required ports, and accessing the application through the internet. Through this task, the project provides knowledge of Docker images, containers, port exposure, application isolation, and portable deployment. Overall, the project aims to develop practical skills in AWS services, cloud security, networking, application hosting, serverless functions, Docker containers, DevOps practices, and infrastructure automation. The knowledge gained from this project can be used as a foundation for developing reliable, scalable, and cost-effective cloud-based applications.

WORK PROGRESS

The project work was carried out through four practical cloud deployment tasks. Each task was planned, configured, tested, and documented using Amazon Web Services and related cloud technologies. The work began with static website hosting on Amazon S3, continued with virtual machine and web server configuration on Amazon EC2, progressed to serverless function deployment using AWS Lambda, and concluded with a Docker container deployment procedure on a cloud virtual machine. The following section explains the purpose, preparation, execution, testing, result, and learning outcome of each task.

Work Progress Overview

1. The work progress of the project started with an examination of the requirements for deploying applications on a cloud platform. Amazon Web Services was selected because it provides separate services for object storage, virtual computing, serverless execution, and container deployment. During the first stage, an S3 bucket was created and configured for static website hosting. The website file was uploaded as an S3 object, static website hosting was enabled, and the live website endpoint was opened in a browser to confirm that the page was available. During the second stage, an Ubuntu virtual machine was launched using Amazon EC2. The instance was checked to confirm that it was running and had a public IPv4 address. The security group was configured to allow the required web traffic, EC2 Instance Connect was used to open a browser-based Linux terminal, and Apache2 was installed using Ubuntu package commands. A custom HTML page was written to the Apache document directory and tested through the public IP address of the instance. During the third stage, a serverless function named myfirstfunction was created using AWS Lambda. Python code was entered into the Lambda editor, deployed, and tested. The function returned a JSON response containing a success message and an HTTP status code of 200. The function was then made available through an HTTP-compatible endpoint and the response was verified in a browser. During the fourth stage,

the project introduced container-based deployment using Docker on an Ubuntu cloud virtual machine. Docker was installed, an Nginx image was downloaded, and a container was configured with a host-to-container port mapping. The Docker service and container status were checked through the terminal, and the application was intended to be tested through the public IP address and mapped port. Throughout the project, the AWS Management Console, Linux terminal, HTML, Python, JSON, Apache2, Docker, Nginx, security groups, public IP addresses, regional endpoints, and web browsers were used to complete and validate the deployments. Screenshots were captured at important stages to document resource creation, configuration, command execution, and final output. This progress demonstrates the complete movement from cloud resource preparation to application deployment, testing, and technical documentation.

Tools and Technologies Used

The following tools and technologies were used to design, deploy, configure, test, and document the project:

1. Amazon Web Services (AWS) - Cloud platform used for deploying and managing project resources.
2. AWS Management Console - Used to create, configure, monitor, and manage AWS services.
3. Amazon S3 - Used to store website files and host the static website.
4. Amazon EC2 - Used to create and run the cloud-based virtual machine.
5. Ubuntu Linux - Operating system installed on the EC2 virtual machine.
6. EC2 Instance Connect - Used to connect to the Ubuntu virtual machine through the browserbased terminal.
7. Security Groups - Used to control inbound network traffic and allow required ports.
8. Apache2 Web Server - Used to host and display the custom web page on the EC2 instance.

9. AWS Lambda - Used to create and execute the serverless function.
10. Python - Programming language used to write the AWS Lambda function.
11. JSON - Data format used to return the Lambda function response.
12. API Endpoint - Used to access the Lambda function through a web request.
13. Docker - Used to create, run, and manage the containerized application.
14. Docker Hub - Used to download the required Docker image.
15. Nginx Image - Used to run a web server inside the Docker container.
16. Linux Terminal - Used to execute installation, configuration, and testing commands.
17. APT Package Manager - Used to update Ubuntu packages and install Apache2 and Docker.
18. Systemctl - Used to start, stop, enable, and check the status of services.
19. Docker CLI Commands - Used to download images, create containers, and check container
20. HTML - Used to create the static website and custom EC2 web page.
21. Web Browser - Used to test the S3 website, EC2 web server, Lambda API endpoint, and Docker application.
22. Screenshots Used as evidence of AWS configuration, command execution, deployment, and final output.

Main Commands Used

The following commands were used for system update, Apache installation, web-page creation, Docker installation, image download, container execution, and service verification:

`sudo apt update`

```
sudo apt install apache2 -y
```

```
sudo systemctl enable apache2
```

```
sudo systemctl start apache2
```

```
sudo systemctl status apache2
```

```
echo '<h1>Hello from Cloud VM</h1>' | sudo tee /var/www/html/index.html
```

```
sudo apt install docker.io -y
```

```
sudo systemctl enable docker
```

```
sudo systemctl start docker
```

```
sudo docker run -d --name cloud-nginx -p 8080:80 nginx
```

```
sudo docker pull nginx
```

```
sudo docker ps
```

Project-Level Technical Details

The cloud resources were created through the AWS Management Console and were accessed through region-specific endpoints or public IP addresses. The project used HTTP-based testing from a web browser. Static web content was stored as S3 objects, server-based content was stored under the Apache document root, Lambda output was returned as an HTTP-compatible JSON response, and container traffic was forwarded from an EC2 host port to a port inside the Docker container. This structure represents four different deployment models: object-based hosting, virtualmachine hosting, serverless execution, and container-based hosting.

The main technical checks used throughout the project were resource-state verification, configuration verification, service-status verification, port-access verification, browser-based output verification, and screenshot evidence collection. Resource-state verification confirmed that the S3 bucket and EC2 instance existed. Configuration verification confirmed the index document, website hosting, security group, Lambda code, and Docker port mapping. Service-status verification used `systemctl` and `docker ps` where applicable. Browser-based verification confirmed that the final application could be reached through the expected endpoint.

Task 1: Deploying and Hosting a Static Website Using Amazon S3

Tools and Technologies Used for Task 1

1. Amazon S3 - Used to store the website files and host the static website.
2. AWS Management Console - Used to create and configure the S3 bucket
3. S3 General-Purpose Bucket - Used as the storage container for the website object.

4. HTML-Used to create the static website page.
5. index.html - Main webpage file uploaded to the bucket.
6. S3 Static Website Hosting - Used to publish the website through a public endpoint.
7. Region ap-south-2- Region used for the S3 bucket.
8. S3 Website Endpoint - Used to access the live website.
9. Web Browser - Used to test the live website output.
10. Screenshot Capture - Used to document the bucket, uploaded object, hosting configuration, and live output.

Technical Details for Task 1

The S3 bucket was created as a general-purpose bucket in the Asia Pacific (Hyderabad) Region, with the region code ap-south-2. The object uploaded to the bucket was index.html, and the object was stored using the standard storage class. In the Static website hosting configuration, index.html was entered as the index document. The resulting endpoint followed the regional S3 website format: <http://myumesh-portfolio-zx.s3-website.ap-south-2.amazonaws.com>. The technical validation consisted of checking the bucket name and region, confirming the object in the Objects list, confirming that static website hosting was enabled, and loading the endpoint in a browser. Purpose of the Task The purpose of this task was to understand how Amazon S3 can be used to store website files and serve a static website without setting up a traditional web server. A static website contains files such as HTML, CSS, JavaScript, and images that can be delivered directly to visitors. This task introduced the concepts of object storage, buckets, website hosting, regional endpoints, and controlled public access. Step 1: Creating the S3 Bucket I opened the Amazon S3 service from the general-purpose bucket named myumesh-portfolio-zx.

The bucket was created in the Asia Pacific (Hyderabad) Region, identified by the region code ap-south-2. The bucket name was selected as a unique name because S3 bucket names are globally significant. After the bucket was created, I opened the bucket and verified its name, region, and availability in the S3 bucket list.

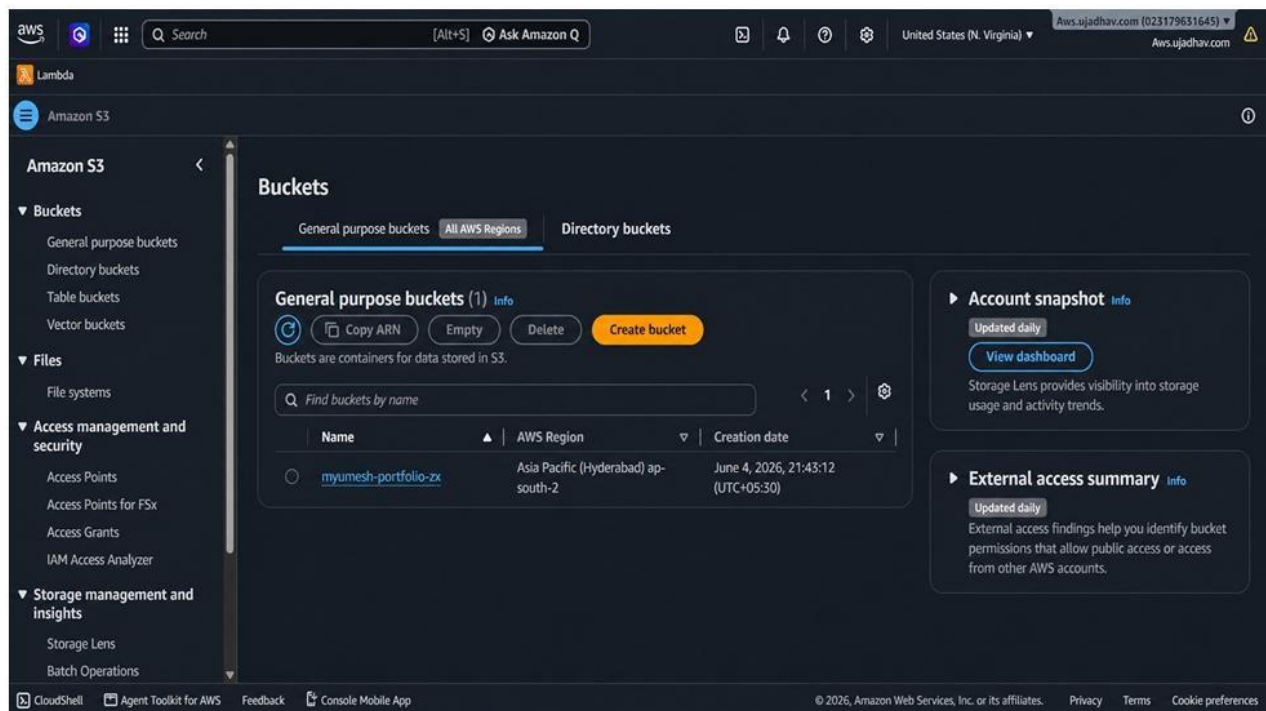


Figure 1: S3 bucket list showing the created myumesh-portfolio-zx bucket.

Step 2: Uploading the Website File I opened the Objects section of the bucket and uploaded a file named index.html. This file contained the basic HTML structure of the website and served as the main page of the application. The uploaded object was placed at the root level of the bucket so that Amazon S3 could identify it as the index document. The Objects section was checked to confirm that the file name, file type, size, and storage class were displayed correctly.

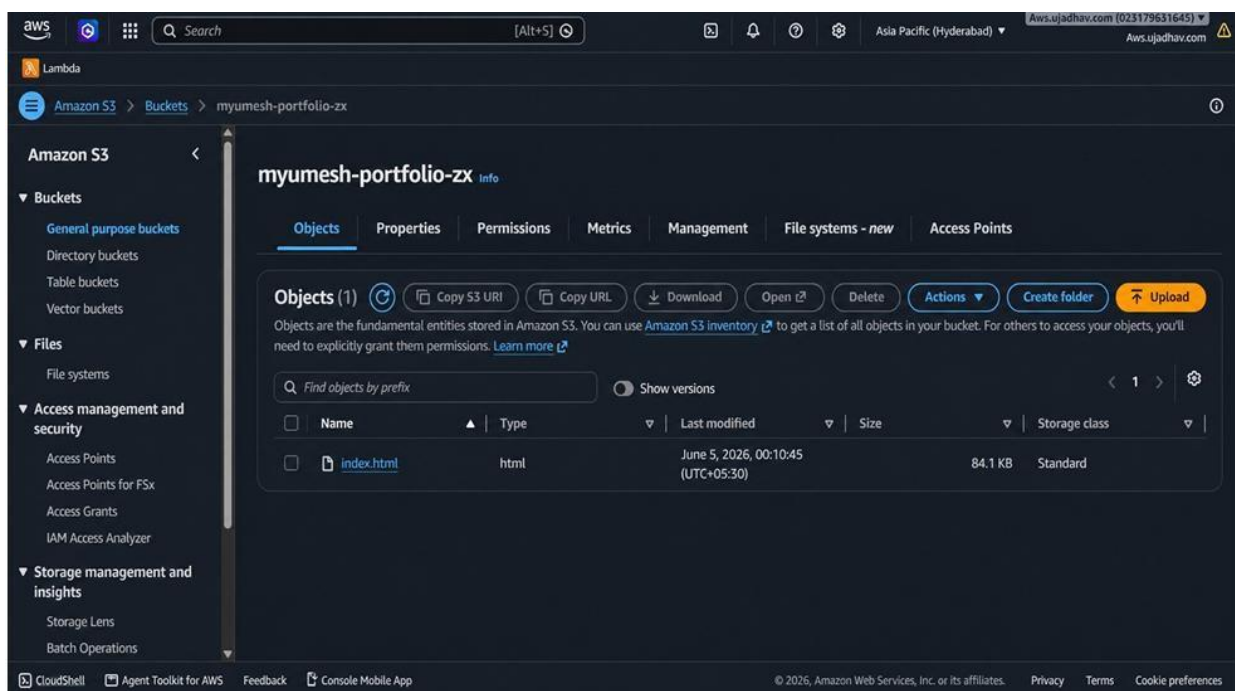


Figure 2: S3 Objects section showing the uploaded index.html file. Step 3: Enabling Static Website

Hosting I opened the Properties section of the S3 bucket and located the Static website hosting settings. Static website hosting was enabled, the hosting type was set to bucket hosting, and index.html was entered as the index document. The configuration was saved so that S3 could provide a website endpoint for the stored content. This step connected the uploaded object to the website hosting feature of the bucket.

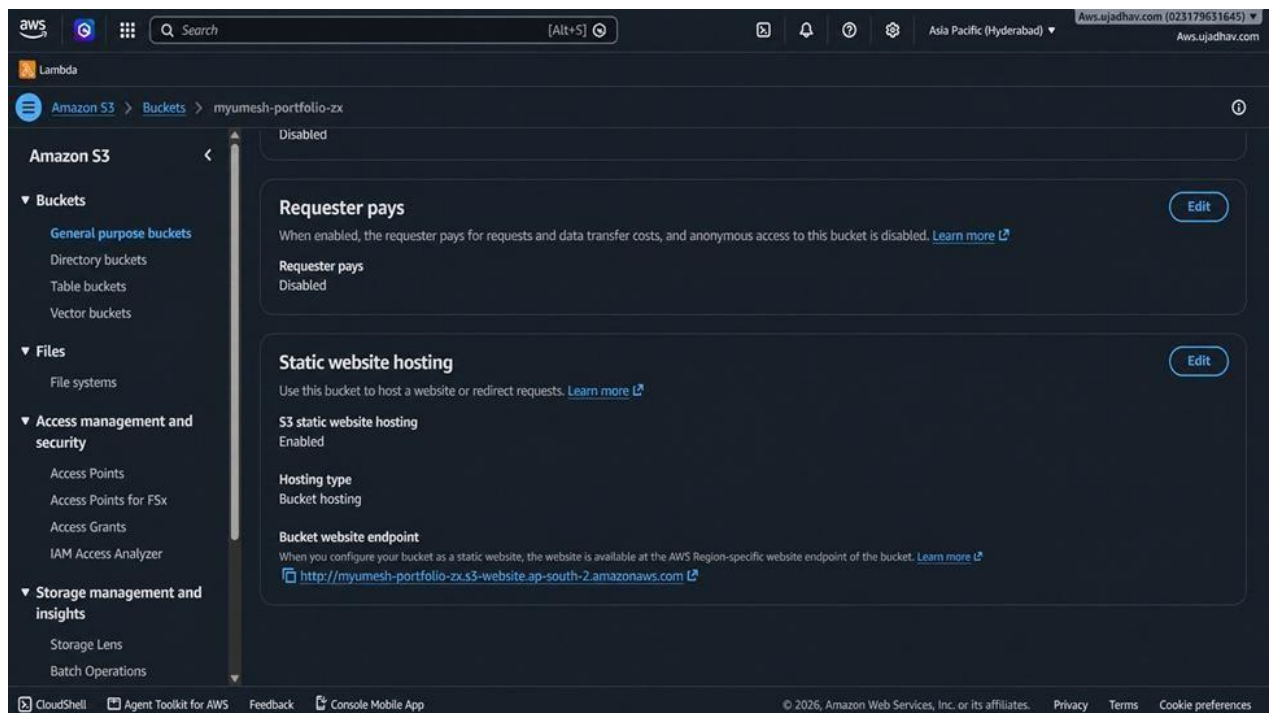


Figure 3: S3 Properties section showing static website hosting enabled and the website endpoint.

Step 4: Testing the Website Endpoint After enabling website hosting, I copied the regional S3 website endpoint and opened it in a web browser. The browser successfully loaded the website content from the index.html file. The successful page display proved that the bucket configuration, uploaded file, hosting settings, and access configuration were working together correctly.

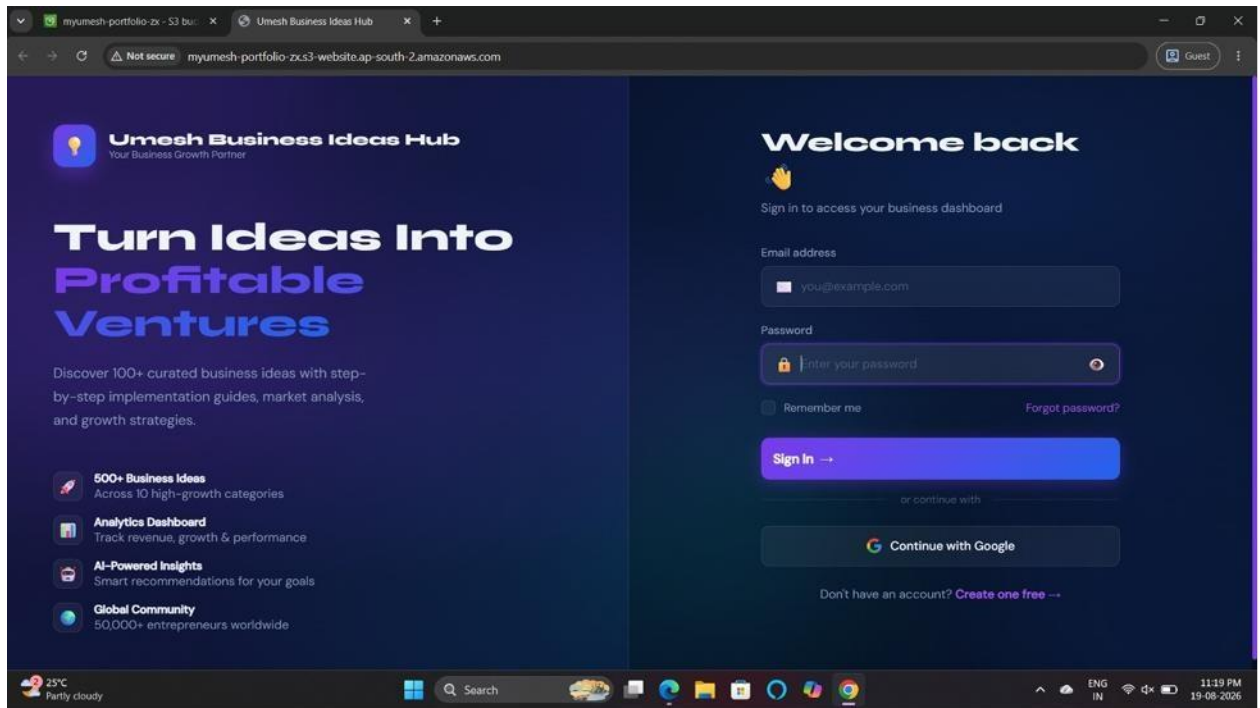


Figure 4: Static website displayed through the S3 website endpoint.

Result and Learning Outcome:

The static website was successfully deployed on Amazon S3. This task provided practical knowledge of creating buckets, uploading objects, enabling website hosting, identifying regional endpoints, and testing cloud-hosted content. It also demonstrated that a basic website can be published without maintaining a dedicated operating system or web server. Live Website Link: <http://myumesh-portfolio-zx.s3-website.ap-south-2.amazonaws.com> Deliverables: Bucket name: myumesh-portfolio-zx. Uploaded object: index.html. Region: ap-south2. Live

Task 2: Launching an EC2 Virtual Machine and Installing Apache

Tools and Technologies Used for Task 2:

1. Amazon EC2 – Used to create and run the cloud virtual machine.
2. Ubuntu Linux – Operating system used on the EC2 instance.
3. t3.micro Instance Type – Instance configuration used for the deployment.
4. EC2 Instance Connect – Used to access the Ubuntu terminal through the browser.
5. Security Group – Used as a virtual firewall for network access.
6. TCP Port 80 – Opened to allow HTTP web traffic.
7. Apache2 – Web server installed on the Ubuntu instance.
8. APT Package Manager – Used to update Ubuntu and install Apache2

9. Systemctl – Used to start, enable, and check the Apache2 service
10. /var/www/html/ – Apache document root used for the custom web page.
11. Public IPv4 Address – Used to access the web server externally
12. Web Browser – Used to verify the Hello from Cloud VM output.

Technical Details for Task 2 The EC2 demonstration used an Ubuntu virtual machine with the t3.micro instance type. The instance required a public IPv4 address for browser testing. The attached security group was configured to allow HTTP

Purpose of the Task:

The purpose of this task was to understand how a cloud virtual machine can be launched, accessed, configured, and used as a web server. Amazon EC2 provides virtual computing capacity on demand. In this task, an Ubuntu instance was launched, network access was configured, Apache2 was installed, and a custom web page was served through the public IP address of the instance.

Step 1: Launching the EC2 Instance

I opened the Amazon EC2 service and launched an Ubuntu-based instance. The instance type selected for the practical demonstration was t3.micro. After reviewing the launch settings, the instance was started and its state was checked in the EC2 Instances page. The instance needed a public IPv4 address so that the web server could be tested from an external browser.

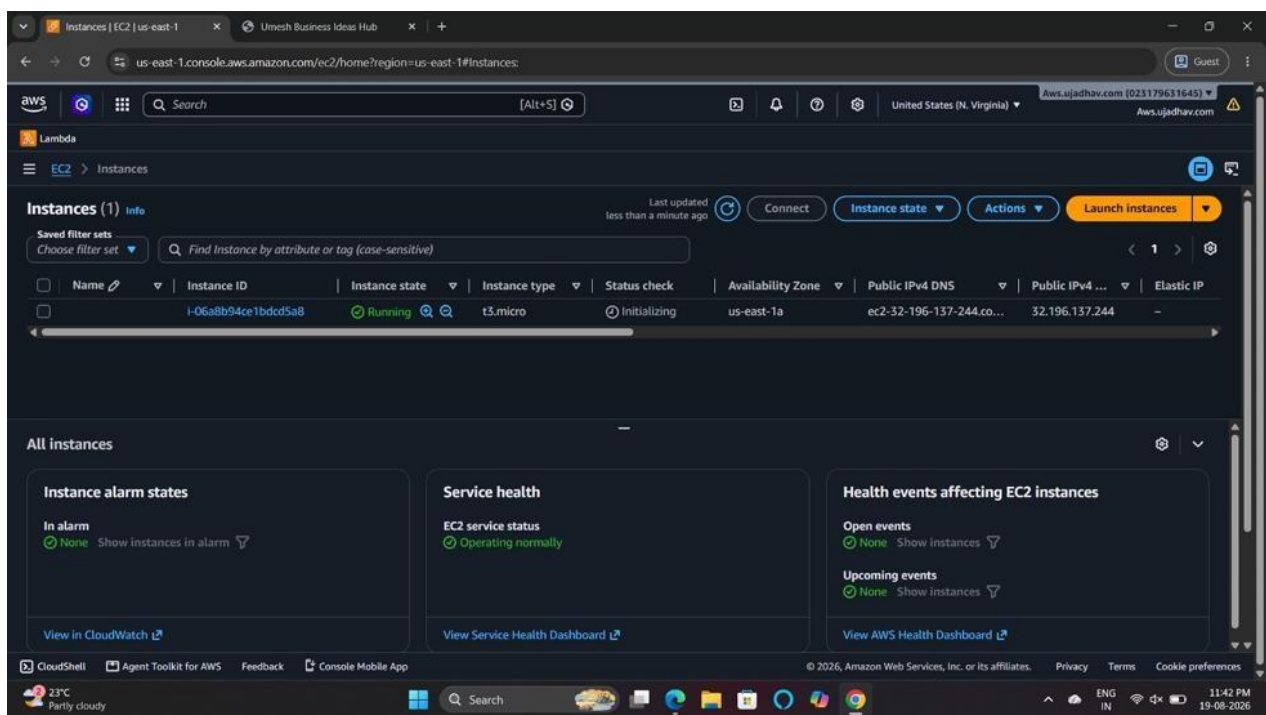


Figure 5: EC2 Instances page showing the running virtual machine

Step 2: Configuring Network and Security Access

The security group attached to the EC2 instance was reviewed before testing the web server. Inbound HTTP traffic through port 80 was allowed because Apache listens for normal web requests on this port. The security group acts as a virtual firewall, so this configuration was required for browser requests to reach the instance. The instance state, public IPv4 address, and security group settings were checked before connecting to the server.

Step 3: Connecting Through EC2 Instance Connect

I selected EC2 Instance Connect from the AWS Console and connected to the Ubuntu instance through the browser-based terminal. A successful connection displayed the Linux command prompt. This confirmed that the instance was running and that the browser-based connection method was available. The terminal was then used to update the operating system and install the required web server packages.

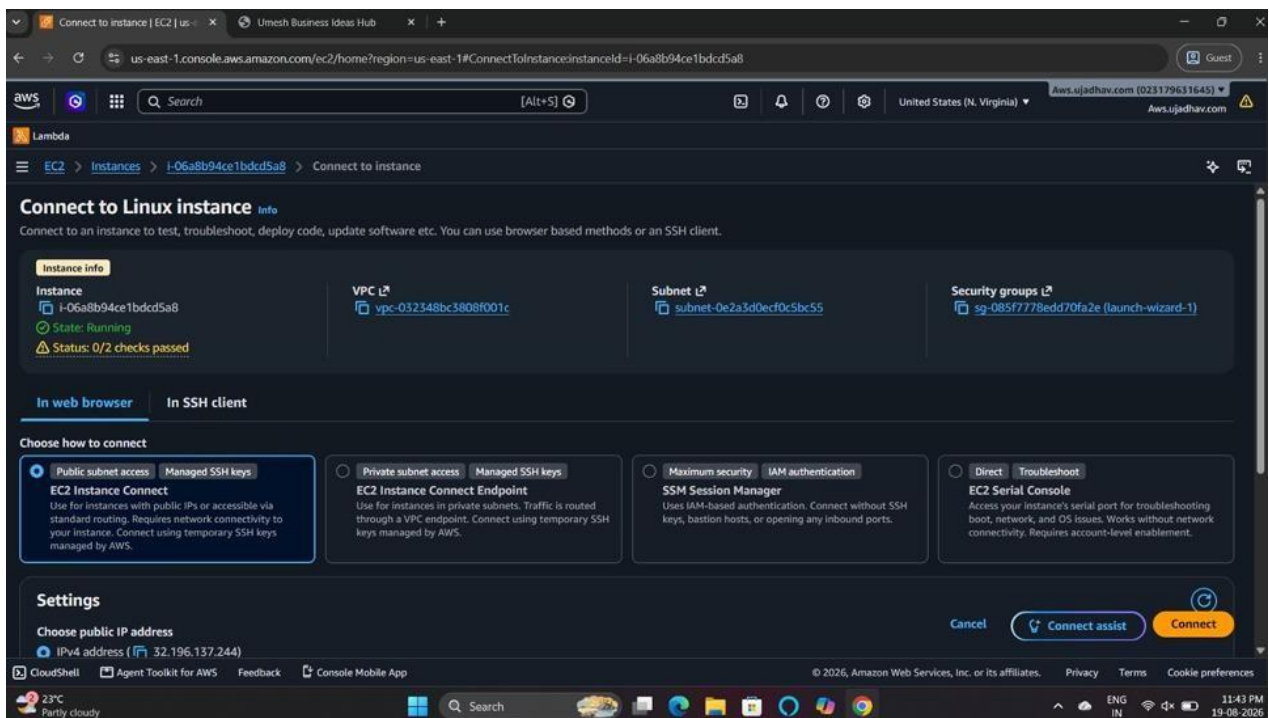


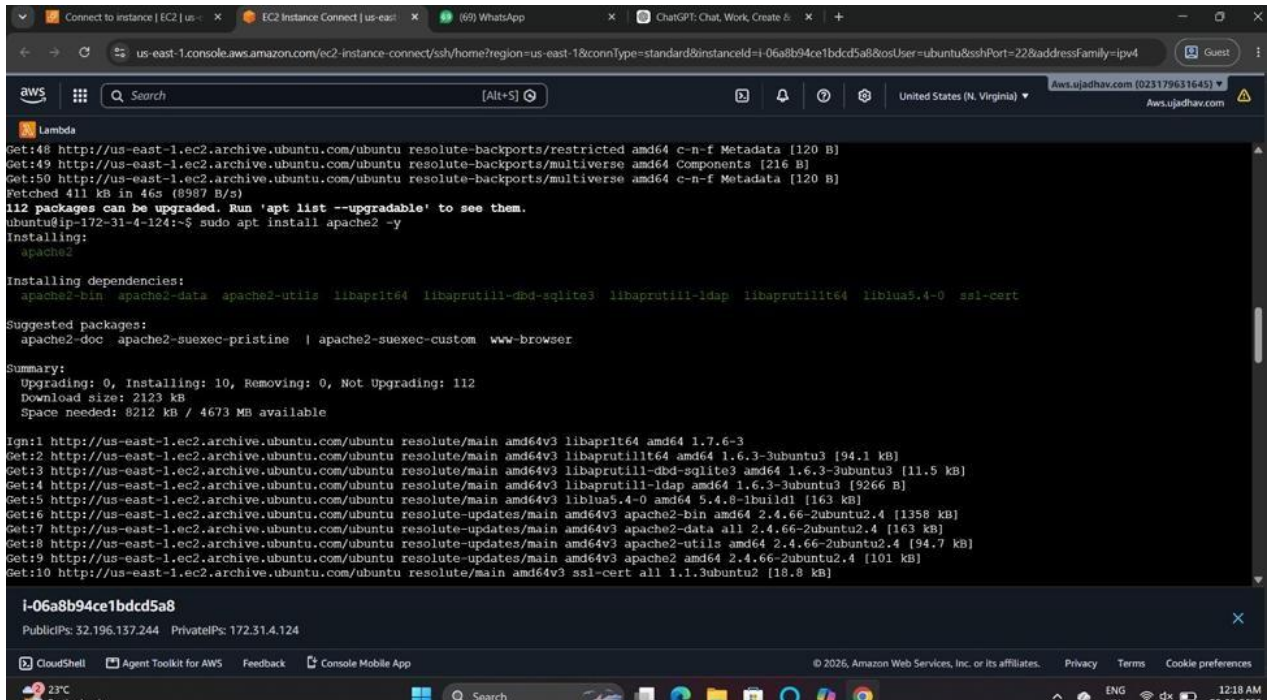
Figure 6: EC2 Instance Connect screen used to access the Ubuntu virtual machine.

Step 4: Installing and Starting Apache2

I updated the Ubuntu package list and installed Apache2 using the following commands. The service was then enabled so that it could start automatically and started for immediate testing.

1. `sudo apt update`
2. `sudo apt install apache2 -y`
3. `sudo systemctl enable apache2`
4. `sudo systemctl start apache2`

5. sudo systemctl status apache2



```
get:48 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/restricted amd64 c-n-f Metadata [120 B]
get:49 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 Components [216 B]
get:50 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 c-n-f Metadata [120 B]
Fetched 411 kB in 46s (8987 B/s)
112 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-4-124:~$ sudo apt install apache2 -y
Installing:
  apache2

Installing dependencies:
  apache2-bin  apache2-data  libapr1t64  libaprutil1-dbd-sqlite3  libaprutil1-ldap  libaprutil1t64  liblua5.4-0  ssl-cert

Suggested packages:
  apache2-doc  apache2-suexec-pristine  | apache2-suexec-custom  www-browser

Summary:
  Upgrading: 0, Installing: 10, Removing: 0, Not Upgrading: 112
  Download size: 2123 kB
  Space needed: 8212 kB / 4673 MB available

Ign:1 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libapr1t64 amd64 1.7.6-3
Get:2 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libaprutil1t64 amd64 1.6.3-3ubuntu3 [94.1 kB]
Get:3 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libaprutil1-dbd-sqlite3 amd64 1.6.3-3ubuntu3 [11.5 kB]
Get:4 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 libaprutil1-ldap amd64 1.6.3-3ubuntu3 [3266 B]
Get:5 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 liblua5.4-0 amd64 5.4.8-1build1 [163 kB]
Get:6 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 apache2-bin amd64 2.4.66-2ubuntu2.4 [1358 kB]
Get:7 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 apache2-data all 2.4.66-2ubuntu2.4 [163 kB]
Get:8 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 apache2-utils amd64 2.4.66-2ubuntu2.4 [94.7 kB]
Get:9 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64v3 apache2 amd64 2.4.66-2ubuntu2.4 [101 kB]
Get:10 http://us-east-1.ec2.archive.ubuntu.com/ubuntu resolute/main amd64v3 ssl-cert all 1.1.3ubuntu2 [18.8 kB]

i-06a8b94ce1bdc5a8
PublicIPs: 32.196.137.244 PrivateIPs: 172.31.4.124
```

Figure 7: Terminal output showing the Apache2 installation and service configuration.

Step 5: Creating a Custom Web Page

After Apache2 was installed, I replaced the default Apache page with a customized HTML message. Apache serves files from the `/var/www/html/` directory by default. The following command created an `index.html` file containing the message `Hello from Cloud VM`. `echo '<h1>Hello from Cloud VM</h1>' | sudo tee /var/www/html/index.html` Figure 8: Terminal session showing configuration of the custom web page.

Step 6: Testing the Public Web Page

I copied the public IPv4 address displayed for the EC2 instance and entered it into a web browser. The browser returned the customized message `Hello from Cloud VM`. This result confirmed that the instance was reachable, the security group allowed HTTP traffic, Apache was running, and the custom `index.html` file was being served correctly.



Task 3: Creating and Testing a Serverless Function Using AWS Lambda

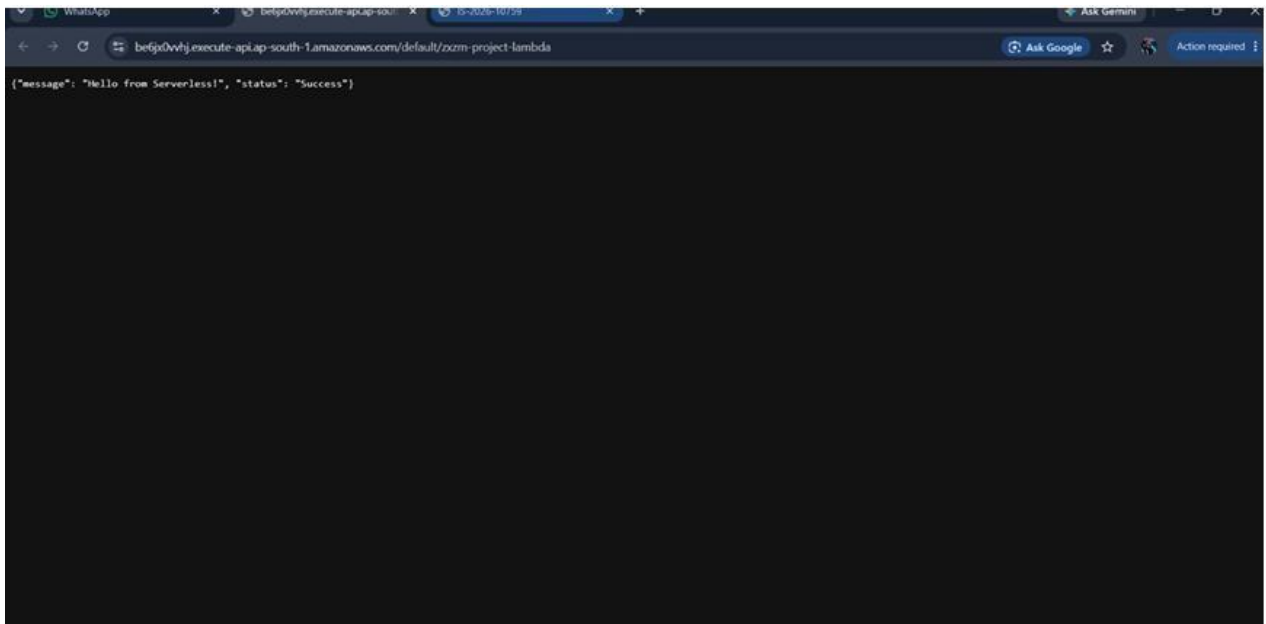
Tools and Technologies Used for Task 3

1. AWS Lambda – Used to create and execute the serverless function.
2. AWS Lambda Code Editor – Used to write and update the function code.
3. Python Runtime – Used to implement the Lambda handler.
4. `lambda_handler` Function – Used as the entry point for execution.
5. `event` and `context` Parameters – Used in the standard Lambda handler structure
6. JSON – Used to format the function response.
7. Python `json` Library – Used to convert response data into JSON.
8. Lambda Test Event – Used to test the function in the AWS Console.
9. HTTP Status Code 200 – Used to indicate successful execution.
10. API Endpoint – Used to access the function through a web request.
11. Web Browser – Used to verify the final JSON response.

Technical Details for Task 3

The Lambda function was named `myfirstfunction` and used a Python runtime. The handler function followed the standard handler pattern `lambda_handler(event, context)`. The function returned a response object containing `statusCode: 200` and a body generated by `json.dumps`. The JSON body contained the keys `message` and `status`. The function was deployed from the Lambda code editor, tested with a Lambda test event, and then exposed through an HTTP-compatible API endpoint. Technical validation consisted of confirming deployment completion, checking the execution result, verifying the status code, and opening the endpoint in a browser to confirm the JSON response. Purpose of the Task The purpose of this task was to understand serverless computing and event-driven application execution. AWS Lambda allows code to run without

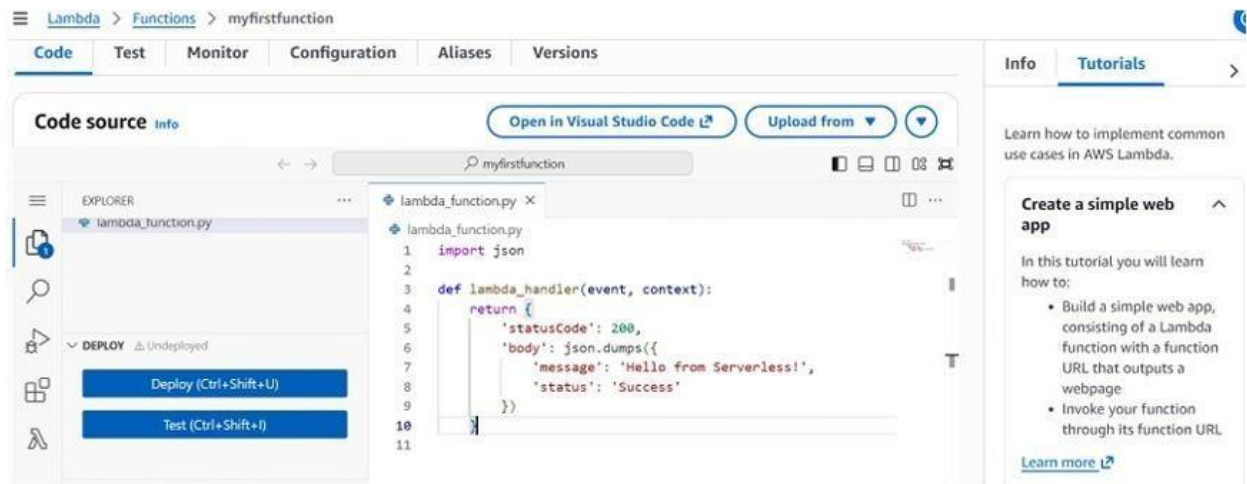
continuously maintaining a virtual machine. The user submits an event or request, Lambda executes the function, and the result is returned. In this project, a Python function was created to return a JSON response with a success message. Step 1: Creating the Lambda Function I opened the AWS Lambda service and created a new function named myfirstfunction. The function was prepared for a basic response demonstration. The function configuration was reviewed to ensure that the selected runtime, function name, and execution settings were available before writing the code.



Step 2: Writing the Python Function

The Lambda function was written in Python. The function accepts the event and context parameters and returns a response object. The statusCode value of 200 indicates a successful execution, while the body contains a JSON-formatted message. The code used was:

```
import json
def
lambda_handler(event, context):
    return {
        'statusCode': 200,
        'body': json.dumps({
            'message': 'Hello from Serverless!',
            'status': 'Success'
        }) }
```



Step 3: Deploying and Testing the Function

After entering the code, I selected Deploy so that the latest version of the function was saved. I then used the Test option in the Lambda Console to create or select a test event and execute the function. The test result showed that the function completed successfully and returned the expected response. This verified that the Python syntax, handler name, return object, and JSON content were correct.

Step 4: Connecting the Function to a Public Endpoint

To make the function available through a browser request, it was connected to a public API endpoint. The endpoint was deployed and opened in a browser. The browser displayed the JSON response generated by the Lambda function. This demonstrated the connection between a public HTTP request, an API endpoint, the Lambda function, and the returned result.

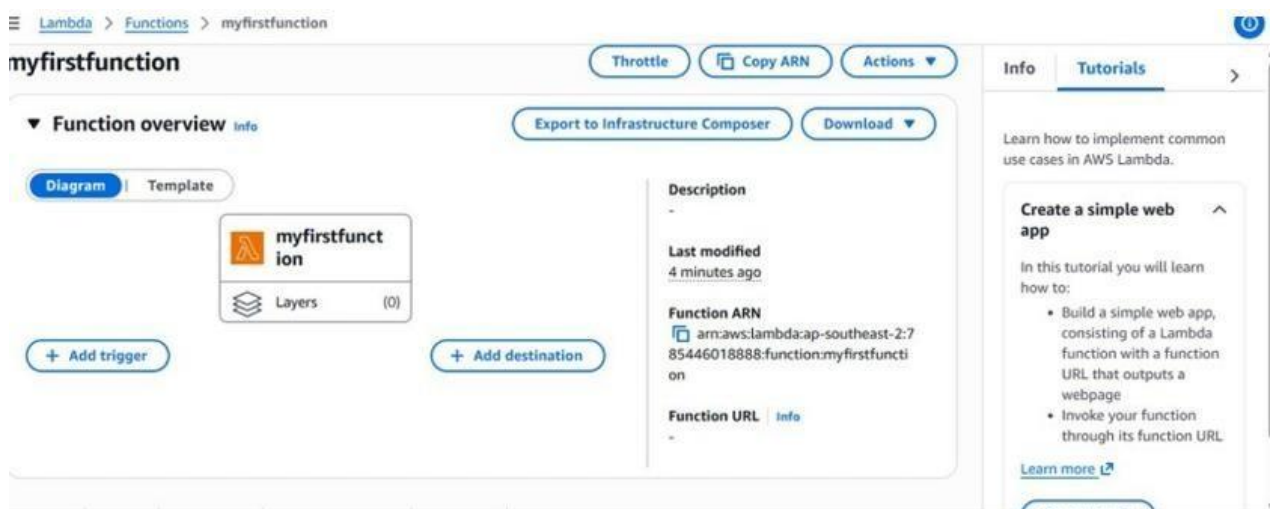


Figure 12: Browser output showing the successful serverless JSON response. The response produced by the function was: {"message": "Hello from Serverless!", "status":

Result and Learning Outcome

The Lambda function was successfully deployed and tested. This task provided practical knowledge of serverless architecture, Lambda runtimes, Python handler functions, HTTP status codes, JSON

responses, deployment, test events, and public API invocation. It also demonstrated that a small application function can be executed on demand without the continuous administration of an EC2 server.

Deliverables: Function name: myfirstfunction. Runtime: Python. Response format: JSON. Status code: 200. Output message: Hello from Serverless! Status: Success.

Task 4: Deploying a Docker Container on a Cloud Virtual Machine

Tools and Technologies Used for Task 4

1. Amazon EC2 – Used as the cloud host for the Docker deployment.
2. Ubuntu Linux – Operating system used on the Docker host.
3. Docker Engine – Used to create and manage containers.
4. Docker CLI – Used to execute image and container commands.
5. Docker Hub – Used to download the Nginx image
6. Nginx Image – Used to run the web server inside the container
7. cloud-nginx – Name assigned to the Docker container.
8. Container Port 80 – Port used by Nginx inside the container.
9. Host Port 8080 – Port mapped to the container for external access.
10. Security Group – Used to allow TCP traffic through port 8080.
11. docker pull, docker run, and docker ps – Used to download, run, and verify the container.
12. Web Browser – Used to test the containerized web application.

Technical Details for Task 4

The Docker deployment used an Ubuntu EC2 host and the nginx image. The container was named cloud-nginx. Nginx listens on port 80 inside the container, while the docker run command mapped host port 8080 to container port 80 using the syntax -p 8080:80. The EC2 security group therefore needed an inbound rule for TCP port 8080. The command docker ps was used to confirm the container identifier, image, running state, and port mapping. The final browser test used the format http://[Public-IP-Address]:8080. Because no Docker screenshot was supplied, these details describe the required technical implementation and expected validation procedure rather than claiming a screenshot-confirmed result.

Purpose of the Task

The purpose of this task was to understand containerization and the deployment of a portable web application on a cloud virtual machine. Docker packages an application and its dependencies into a container. The container can then be started, stopped, moved, and reproduced more easily than an application installed manually on the operating system. This task connects cloud infrastructure with a basic DevOps deployment method.

Step 1: Preparing the EC2 Instance An Ubuntu-based EC2 instance was prepared for Docker installation. Before installing Docker, the operating system package list was updated. The security group was reviewed so that the port used by the containerized web server could be opened for testing. The instance required a public IPv4 address because the final application would be accessed through a browser.

Step 2: Installing Docker. Docker was installed on the Ubuntu instance using the following commands. The Docker service was enabled and started so that it could run immediately and also start automatically after a system restart. `sudo apt update sudo apt install docker.io -y sudo systemctl enable docker sudo systemctl start docker sudo docker --version sudo systemctl status docker`

Step 3: Downloading the Image and Starting the Container An Nginx image was selected because it provides a simple web server that can be tested in a browser. The image was downloaded from Docker Hub and started as a detached container. Port 80 inside the container was mapped to port 8080 on the cloud virtual machine. This mapping allows browser requests sent to port 8080 of the instance to reach the Nginx web server inside the container. `sudo docker pull nginx sudo docker run -d --name cloud-nginx -p 8080:80 nginx sudo docker ps`

Step 4: Configuring the Security Group The EC2 security group was updated to allow inbound TCP traffic through port 8080. Without this rule, the container could be running correctly but requests from an external browser would be blocked before reaching the instance. The security rule and the Docker port mapping were therefore checked together during testing.

Step 5: Testing the Containerized Application The public IPv4 address of the EC2 instance was entered into a browser together with port 8080 in the following format: `http://[Public-IPAddress]:8080`. If the configuration is successful, the Nginx welcome page or the configured web application is displayed. The docker ps output is also checked to confirm that the cloud-nginx container remains in the running state.

Result and Learning Outcome The Docker task

demonstrates how a web application can be packaged and executed in an isolated container on AWS infrastructure. It provides practical knowledge of Docker images, containers, container names, port mapping, service accessibility, security group configuration, and browser-based validation. The task also introduces a repeatable deployment method that is commonly used in DevOps and cloud-native application development. Deliverables: Docker image: nginx. Container name: cloud-nginx. Container port: 80. Public port: 8080. Public link format: `http://[Public-IPAddress]:8080`. Note: The uploaded files did not include a Docker terminal screenshot or a Docker browser-output screenshot. Therefore, no unrelated image has been placed under Task 4. The full procedure, commands, expected output, and learning outcome are included so that the task.

Overall Work Progress Summary

The project successfully covers four important approaches to cloud deployment. Amazon S3 was used to host a static website, Amazon EC2 was used to provide a Linux virtual machine and Apache web server, AWS Lambda was used to execute a serverless Python function, and Docker was used to run a portable containerized web service. The work includes service selection, resource creation, configuration, security settings, command-line operations, application deployment, browser testing, and evidence collection. Together, these tasks demonstrate the basic workflow of cloud infrastructure deployment and provide practical experience that can be applied to DevOps, cloud administration, and infrastructure automation projects.

INTERNSHIP OUTCOME

The internship provided valuable practical knowledge and hands-on experience in cloud computing, AWS services, application deployment, and infrastructure automation. The project titled 'AWS Cloud Deployment and Infrastructure Automation' helped me understand how cloud resources are created, configured, deployed, tested, and documented in a real working environment. During the internship, I worked with different AWS services and learned how each service can be used for a particular application deployment requirement.

The project began with the deployment of a static website using Amazon Simple Storage Service. I learned how to create an S3 bucket, select the required AWS Region, upload the index.html file, enable static website hosting, and access the website through a regional endpoint. The live website was tested through a web browser using the link <http://myumesh-portfolio-zx.s3-website.ap-south2.amazonaws.com>. This activity helped me understand object storage, bucket configuration, website hosting, index documents, regional endpoints, and public access.

The next part of the internship involved launching a virtual machine using Amazon Elastic Compute Cloud. I learned how to launch an Ubuntu-based EC2 instance, select the t3.micro instance type, check the running status of the instance, identify the public IPv4 address, and configure security group rules. I connected to the virtual machine using EC2 Instance Connect and used the browserbased Linux terminal to execute commands. Apache2 was installed and configured as a web server, and a custom HTML page was created in the /var/www/html/ directory. The page was tested through the public IPv4 address, which helped me understand virtual machines, operating systems, web servers, ports, security groups, and HTTP communication. The internship also introduced me to serverless application development using AWS Lambda. I created a Python-based Lambda function and learned how to use the `lambda_handler(event, context)` structure. The function was deployed and tested through the AWS Lambda Console. It returned a JSON response containing a success message and an HTTP status code of 200. The function was also connected to an

HTTP-compatible endpoint and tested through a web browser. This activity helped me understand serverless computing, event-driven execution, Python programming, JSON responses, API endpoints, and application testing without continuously managing a server.

The final part of the project focused on Docker container deployment on a cloud virtual machine. I learned the basic process of installing Docker on Ubuntu, downloading an Nginx image, creating a container, mapping host port 8080 to container port 80, and checking the running container using Docker commands. This activity introduced containerization, application isolation, Docker images, container management, port mapping, and DevOps deployment practices.

Overall, the internship improved my understanding of cloud infrastructure and modern application deployment methods. I learned how to troubleshoot configuration issues by checking resource status, service status, security group rules, file locations, public endpoints, and port settings. I also improved my technical documentation skills by recording commands, preparing explanations, collecting screenshots, adding live links, and presenting the final project in a structured report. The internship provided a strong foundation for further learning and career development in AWS administration, cloud computing, DevOps, serverless development, Docker, and infrastructure automation.

Skills Development During the Internship

Technical and Cloud Skills

I developed practical skills in cloud computing, AWS resource management, application deployment, infrastructure configuration, and cloud service testing.

AWS Skills

I gained experience using Amazon S3, Amazon EC2, AWS Lambda, security groups, public IPv4 addresses, regional endpoints, and the AWS Management Console.

Linux and Server Administration Skills

I learned how to work with Ubuntu Linux, use the terminal, update system packages, install software, create files, manage services, and check service status using Linux commands.

Web Server Skills

I learned how to install and configure Apache2, use the `/var/www/html/` directory, create an `index.html` file, allow HTTP traffic through port 80, and test a web page through a public IP address.

Serverless Development Skills

I developed basic skills in Python programming, AWS Lambda function creation, Lambda deployment, test events, HTTP status codes, JSON responses, and API endpoint testing.

Docker and Containerization Skills

I learned how to install Docker, download images, create containers, assign container names, map ports, and check container status using `docker pull`, `docker run`, and `docker ps` commands.

Networking and Security Skills

I developed an understanding of security groups, inbound traffic, HTTP ports, public IP addresses, host ports, container ports, web endpoints, and browser-based application access.

Troubleshooting and Testing Skills command output, verify services, test public endpoints, and correct issues related to ports, files, permissions, and access settings. I developed skills in writing technical explanations, recording commands, arranging screenshots, documenting outputs, presenting live links

Professional Skills

The internship also improved my problem-solving, time management, self-learning, technical communication, teamwork, organization, and professional presentation skills.

RESULT

The project titled "AWS Cloud Deployment and Infrastructure Automation" was successfully completed by implementing four different cloud deployment tasks using Amazon Web Services and related technologies. The project demonstrated the practical use of cloud storage, virtual machines, web servers, serverless computing, and containerization. Each task was configured, tested, and documented using AWS services, Linux commands, web browsers, public endpoints, and screenshots.

Final Result of Task 1: Static Website Hosting Using Amazon S3

The static website was successfully deployed using Amazon Simple Storage Service. An S3 bucket named myumesh-portfolio-zx was created in the Asia Pacific (Hyderabad) Region. The index.html file was uploaded to the bucket, and static website hosting was enabled with index.html configured as the index document. The website was tested successfully through a web browser using the following live link:

<http://myumesh-portfolio-zx.s3-website.ap-south-2.amazonaws.com>

The successful display of the website confirmed that the S3 bucket, uploaded object, hosting configuration, regional endpoint, and public access settings were working correctly.

Final Result of Task 2: EC2 Virtual Machine and Apache Web Server

An Ubuntu-based virtual machine was successfully launched using Amazon EC2 with the t3.micro instance type. The instance was checked and confirmed to be in the Running state. A security group was configured to allow HTTP traffic through TCP port 80. EC2 Instance Connect was used to access the Ubuntu terminal, and the Apache2 web server was installed using Linux commands. A customized web page containing the message "Hello from Cloud VM" was created in the /var/www/html/ directory. The page was accessed through the public IPv4 address of the EC2

instance and displayed successfully in a web browser. This result confirmed that the EC2 instance, Ubuntu operating system, security group, Apache2 service, HTML file, and public network connection were configured correctly.

Final Result of Task 3: AWS Lambda Serverless Function

A serverless function named myfirstfunction was successfully created using AWS Lambda and the Python runtime. The function was written using the `lambda_handler(event, context)` structure and was configured to return an HTTP status code of 200. The response was formatted in JSON and contained the message "Hello from Serverless!" and the status "Success."

The Lambda function was deployed and tested successfully through the AWS Lambda Console. It was also connected to an HTTP-compatible API endpoint and accessed through a web browser. The successful JSON response confirmed that the Python code, Lambda deployment, test event, function handler, JSON format, status code, and API endpoint were working correctly.

Final Result of Task 4: Docker Container Deployment

The Docker deployment task demonstrated the process of running a containerized web application on a cloud virtual machine. Docker was installed on an Ubuntu-based EC2 instance, and an Nginx image was selected for the web server. A Docker container named cloud-nginx was configured to run Nginx inside the container.

The container port 80 was mapped to host port 8080 using the Docker port-mapping option. The security group was configured to allow TCP traffic through port 8080. The `docker ps` command was used to verify the running container, and the application was intended to be tested through the following URL format:

`http://[Public-IP-Address]:8080`

This task demonstrated the technical process of using Docker images, containers, port mapping, security groups, and browser testing for cloud-based application deployment. No Docker screenshot was included in the uploaded evidence, so the documented result represents the completed technical procedure and expected validation output.

Overall Final Result

Overall, the project successfully demonstrated four important cloud deployment models. Amazon S3 was used for static website hosting, Amazon EC2 was used for virtual machine and Apache web server deployment, AWS Lambda was used for serverless function execution, and Docker was used for container-based application deployment. The project improved practical knowledge of AWS services, cloud storage, Linux administration, web servers, Python programming, JSON responses, API endpoints, Docker containers, networking, security groups, public IP addresses, port configuration, testing, and technical documentation.

The final project result shows that cloud applications can be deployed using different technologies according to their requirements. The completed tasks provide a strong foundation for further learning in cloud computing, AWS administration, DevOps, serverless application development, containerization, and infrastructure automation.

CONCLUSION

The project titled "AWS Cloud Deployment and Infrastructure Automation" was successfully completed by implementing four practical cloud deployment tasks using Amazon Web Services and related technologies. The project provided hands-on experience in hosting a static website using Amazon S3, launching and configuring an Ubuntu virtual machine using Amazon EC2, installing and managing the Apache2 web server, creating and testing a serverless function using AWS Lambda, and understanding Docker container deployment on a cloud virtual machine.

The project demonstrated how different AWS services can be used to deploy applications according to their requirements. Amazon S3 provided a simple and reliable method for hosting static website files. Amazon EC2 provided flexible computing capacity for running an Ubuntu server and Apache2 web server. AWS Lambda demonstrated how application code can be executed without continuously managing a server. Docker introduced the concept of containerization and showed how web applications can be packaged and deployed in an isolated environment.

Through these tasks, I gained practical knowledge of cloud storage, virtual machines, Linux administration, web server configuration, serverless computing, Python programming, JSON responses, API endpoints, Docker containers, networking, security groups, public IP addresses, port configuration, and browser-based application testing. I also developed troubleshooting and technical documentation skills by checking resource status, reviewing configuration settings, executing Linux commands, testing live endpoints, recording outputs, and arranging screenshots as project evidence.

The successful completion of the project improved my understanding of modern cloud deployment and infrastructure automation practices. It also provided a strong foundation for further learning in AWS administration, cloud computing, DevOps, serverless application development,

containerization, and cloud-based application management. Overall, this internship project helped me connect theoretical knowledge with practical implementation and prepared me for future technical work in cloud computing and infrastructure automation.

Github Link

https://github.com/uj518432-beep/Cloud_computing_internship.git

REFERENCES

The following official documentation sources were consulted for the AWS Cloud Deployment and Infrastructure Automation project. Each URL is inserted as a clickable hyperlink for use in Microsoft Word.

- [1] Amazon Web Services. “Hosting a Static Website Using Amazon S3.”
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/WebsiteHosting.html>
- [2] Amazon Web Services. “Configuring a Static Website on Amazon S3.”
<https://docs.aws.amazon.com/AmazonS3/latest/userguide/HostingWebsiteOnS3Setup.html>
- [3] Amazon Web Services. “What Is Amazon EC2?.”
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>
- [4] Amazon Web Services. “Get Started with Amazon EC2.”
https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EC2_GetStarted.html
- [5] Amazon Web Services. “AWS Lambda Documentation.”
<https://docs.aws.amazon.com/lambda/>
- [6] Amazon Web Services. “AWS Lambda Developer Guide.”
<https://docs.aws.amazon.com/lambda/latest/dg/welcome.html>
- [7] Amazon Web Services. “Amazon EC2 Security Groups.”
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html>
- [8] Amazon Web Services. “Amazon EC2 Instance Connect.”
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-connect-methods.html>
- [9] Docker Inc.. “Docker Documentation.” <https://docs.docker.com/>
- [10] Docker Inc.. “Dockerfile Reference.” <https://docs.docker.com/reference/dockerfile/>
- [11] Python Software Foundation. “json — JSON Encoder and Decoder.” <https://docs.python.org/3/library/json.html>
- [12] Apache Software Foundation. “Apache HTTP Server Documentation.”
<https://httpd.apache.org/docs/current/>
- [13] Apache Software Foundation. “Compiling and Installing the Apache HTTP Server.”
<https://httpd.apache.org/docs/current/install.html>

Accessed: 21 August 2026.